



**UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE – CAMPUS NATAL CURSO  
TECNOLOGIA DA INFORMAÇÃO**

**RELATÓRIO: Trabalho Prático 1 Und. 2 – Tipos Abstratos de Dados (TADs)**

**Aluno: Nelson Ryan Silva de Vasconcelos**

**Matrícula: 20250062976**

**Disciplina: IMD0029 - ESTRUTURA DE DADOS BÁSICAS I - T06 (2026.1)**

**Trabalho Unidade II :Trabalho Prático 1 Und. 2 – Tipos Abstratos de Dados (TADs)**

**Professora: Ligia Francielle Borges**

**Data: 26/05/2026**

**Repositório GitHub:**

**<https://github.com/ryanvasconcelos/projeto-tads-nelson-vasconcelos-edb1-2026.1.git>**

## 1. Introdução

Este relatório documenta o desenvolvimento do Trabalho Prático 1 da disciplina de Estruturas de Dados Básicas I (IMD0029). O objetivo principal do projeto foi implementar três Tipos Abstratos de Dados (TADs) distintos utilizando a linguagem C++: uma Sequência, um Conjunto e um Array Sequence.

O desenvolvimento seguiu rigorosamente os requisitos exigidos, garantindo a modularização do código (separação em arquivos .h e .cpp) e o uso exclusivo de alocação dinâmica de memória via ponteiros. O uso de estruturas prontas da STL (Standard Template Library) não foram utilizadas, com o intuito de aplicar na prática os conceitos fundamentais de gerenciamento de memória, controle de escopo e movimentação de vetores em baixo nível.

## 2. Descrição teórica de cada TAD

**TAD Sequência:** Trata-se de uma estrutura de dados linear baseada em um vetor dinâmico com capacidade pré-definida. A sua característica principal é o acesso, inserção e remoção de elementos baseados em índices. Para manter a adjacência dos dados, as operações de inserção e remoção exigem o deslocamento dos elementos adjacentes na memória.

**TAD Conjunto:** É uma estrutura relacionada na teoria dos conjuntos da Matemática Discreta. A sua regra primária é a não aceitação de elementos duplicados. Foi implementada sobre um vetor não ordenado, exigindo buscas lineares de verificação antes de qualquer adição. A estrutura também suporta operações matemáticas de União e Interseção, que operam sem modificar os conjuntos originais, instanciando e retornando novos conjuntos na memória.

**Array Sequence:** Consiste em uma evolução da estrutura sequencial padrão, incorporando a capacidade de redimensionamento dinâmico automático. O seu diferencial teórico é a gestão invisível de memória: quando a capacidade máxima do vetor atual é atingida, a estrutura aloca um novo bloco de memória na heap com o dobro do tamanho original, transfere os dados com segurança e desaloca o bloco antigo, prevenindo erros de transbordamento (overflow).

## 3. Explicação das principais operações

### TAD Sequência e Array Sequence

- **insert(pos, elem):** No melhor cenário (inserir no final da estrutura), a operação é executada em tempo constante,  $O(1)$ . No entanto, no pior cenário (inserir na posição 0), todos os elementos existentes precisam de ser deslocados uma posição para a direita através de um laço for. Portanto, a complexidade no pior caso é  $O(n)$ , onde  $n$  é o número de elementos atuais.
- **remove(pos):** Semelhante à inserção, remover o último elemento é  $O(1)$ . Remover o primeiro elemento obriga a reorganizar todo o vetor, puxando os elementos restantes uma casa para a esquerda. O pior caso é  $O(n)$ .
- **get(pos):** O acesso a qualquer elemento através do seu índice é direto e imediato, resultando numa complexidade de  $O(1)$ .
- **realocar() (Array Sequence):** Esta operação faz a alocação de um novo bloco de memória e copia os  $n$  elementos antigos para o novo espaço. Por envolver uma cópia completa, a sua complexidade é  $O(n)$ . Consequentemente, o pushBack passa

a ter uma complexidade  $O(1)$  amortizada (é quase sempre instantâneo, exceto quando precisa de realocar).

- **dimension():** Retorna a capacidade total de memória atualmente alocada para o vetor dinâmico. Por ser apenas uma leitura direta de uma variável inteira (capacidade), a sua execução é instantânea, com complexidade de  $O(1)$ .

### TAD Conjunto

- **contains(elem):** Como o vetor utilizado não está ordenado, a única forma de verificar a existência de um elemento é percorrendo-o do início ao fim (Busca Linear). No pior caso, o elemento não existe ou está na última posição, gerando uma complexidade de  $O(n)$ .
- **add(elem):** Antes de adicionar, a estrutura chama obrigatoriamente a função **contains**. Como a busca linear custa  $O(n)$  e a inserção no fim custa  $O(1)$ , o custo total da operação de adição é dominado pela busca, sendo  $O(n)$ .
- **uniao(A, B) e intersecao(A, B):** Ambas as operações precisam de comparar e cruzar dados entre dois conjuntos. A união percorre ambos os conjuntos chamando **add()** (que por sua vez faz uma busca), e a interseção percorre o conjunto A verificando se cada elemento existe em B através do **contains()**. Se ambos os conjuntos tiverem tamanho  $n$ , o tempo total de execução no pior cenário será de  $O(n^2)$ .

### 5. Tabela de Complexidade das Operações

A tabela abaixo mostra a complexidade temporal de cada uma das operações obrigatórias exigidas no projeto, considerando o pior caso.

| Estrutura (TAD) | Operação          | Complexidade (Pior Caso) | Justificação Detalhada                     |
|-----------------|-------------------|--------------------------|--------------------------------------------|
| TAD Sequência   | insert(pos, elem) | $O(n)$                   | Deslocamento de elementos para a direita.  |
|                 | remove(pos)       | $O(n)$                   | Deslocamento de elementos para a esquerda. |

|                     |                  |                      |                                                                     |
|---------------------|------------------|----------------------|---------------------------------------------------------------------|
|                     | get(pos)         | $O(1)$               | Acesso direto por índice na memória heap.                           |
|                     | print()          | $O(n)$               | Laço para percorrer e imprimir todos os elementos.                  |
| <b>TAD Conjunto</b> | add(elem)        | $O(n)$               | Custo da busca linear prévia com contains.                          |
|                     | remove(elem)     | $O(n)$               | Busca linear para achar o elemento + preenchimento do espaço vazio. |
|                     | contains(elem)   | $O(n)$               | Busca linear em vetor não ordenado.                                 |
|                     | uniao(A, B)      | $O(n*m)$ ou $O(n^2)$ | Inserção de todos os elementos com verificação de duplicados.       |
|                     | intersecao(A, B) | $O(n*m)$ ou $O(n^2)$ | Verificação de cada elemento de A dentro do conjunto B.             |

|                       |                    |                   |                                                                 |
|-----------------------|--------------------|-------------------|-----------------------------------------------------------------|
| <b>Array Sequence</b> | pushBack(value)    | $O(1)$ Amortizado | Inserção direta no fim, exceto quando há realocação ( $O(n)$ ). |
|                       | pushFront(value)   | $O(n)$            | Obriga sempre ao deslocamento de todos os elementos.            |
|                       | insert(pos, value) | $O(n)$            | Deslocamento proporcional à posição escolhida.                  |
|                       | remove(pos)        | $O(n)$            | Remoção com deslocamento para a esquerda.                       |
|                       | find(value)        | $O(n)$            | Busca linear para identificar o índice do valor.                |
|                       | size()             | $O(1)$            | Retorno direto da variável inteira tamanho_atual.               |
|                       | dimension()        | $O(1)$            | Retorno direto da variável inteira capacidade.                  |

#### 4. Trechos relevantes do código

Abaixo, destacam-se os trechos de código que representam partes relevantes das estruturas implementadas:

**1. Lógica de Redimensionamento (Array Sequence):** O método `realocar()` demonstra a gestão manual da memória na heap. Quando a capacidade é atingida, um novo bloco com o dobro do tamanho é alocado, os dados são transferidos de forma segura e o bloco antigo é destruído para evitar memory leaks.

```
15 void ArraySequence::realocar(){
16
17     capacidade *= 2;
18     std::cout << "A V I S O: Array foi realocado e redimensionado com a capacidade: " << capacidade << std::endl;
19
20     int* novo_array = new int[capacidade];
21
22     for(int i = 0; i < tamanho_atual; i++){
23         novo_array[i] = elementos[i];
24     }
25
26     delete[] elementos;
27
28     elementos = novo_array;
29 }
```

**2. Lógica de Interseção e Gestão de Escopo (TAD Conjunto):** A função amiga (friend) `intersecao` e (friend) `uniao` ilustra a passagem de objetos por referência para evitar cópias desnecessárias e a instanciação de um novo conjunto cujo tamanho é otimizado com base na menor capacidade entre os dois conjuntos avaliados.

```
66 Conjunto* uniao(Conjunto& A, Conjunto& B){
67     //Novo conjunto para armazenar a uniao de dois vetores
68     Conjunto* novo_conjunto = new Conjunto(A.capacidade + B.capacidade);
69
70     for(int i = 0; i < A.tamanho_atual; i++){
71         novo_conjunto->add(A.elementos[i]);
72     }
73
74     for(int i = 0; i < B.tamanho_atual; i++){
75         novo_conjunto->add(B.elementos[i]);
76     }
77
78     return novo_conjunto;
79 }
```

```
81 Conjunto* intersecao(Conjunto& A, Conjunto& B){
82     int menor_capacidade;
83     //Verificacao para menor capacidade da intersecao
84     if (A.tamanho_atual < B.tamanho_atual) {
85         menor_capacidade = A.capacidade;
86     } else {
87         menor_capacidade = B.capacidade;
88     }
89     //Novo conjunto para armazenar a intersecao de dois conjuntos
90     Conjunto* novo_conjunto = new Conjunto(menor_capacidade);
91
92     for(int i = 0; i < A.tamanho_atual; i++){ //Laco que adiciona os elementos em comum ao novo conjunto
93         if(B.contains(A.elementos[i])){
94             novo_conjunto->add(A.elementos[i]);
95         }
96     }
97     return novo_conjunto;
98 }
```

## 6. Evidências dos testes realizados

Figura 1 - Execução do programa de teste do TAD Sequência

```
nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/tad-sequencia
$ make
g++ -Wall -std=c++17 -c main.cpp
g++ -Wall -std=c++17 -c sequencia.cpp
g++ -Wall -std=c++17 -o exec_sequencia main.o sequencia.o

nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/tad-sequencia
$ make run
./exec_sequencia
----- TAD SEQUENCIAL: TESTE -----

Inserindo 10 elementos na sequencia . . .
Sequencia original:
[10] [20] [30] [40] [50] [60] [70] [80] [90] [100]

Removendo 2 elementos do meio (posicao 4 e posicao 5) . . .
Sequencia apos a remocao:
[10] [20] [30] [40] [70] [80] [90] [100]

Novos elementos das posicoes 4 e 5, apos remocao:
70 e 80
```

Figura 2 - Execução do programa de teste do TAD Conjunto

```
nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/tad-conjunto
$ make
g++ -Wall -std=c++17 -c main.cpp
g++ -Wall -std=c++17 -c conjunto.cpp
g++ -Wall -std=c++17 -o exec_conjunto main.o conjunto.o

nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/tad-conjunto
$ make run
./exec_conjunto
----- TAD CONJUNTO: TESTE -----

Seja o conjunto A: { 1, 2, 3, 5 }
Seja o conjunto B: { 3, 4, 5, 6 }

A uniao eh dada por: { 1, 2, 3, 5, 4, 6 }

A intersecao eh dada por: { 3, 5 }

Removendo o elemento 2 que esta na posicao 1 de A:
{ 1, 2, 3, 5 }
```

Figura 3 - Execução do programa de teste do Array Sequence

```
nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/array-sequence
$ make
g++ -Wall -std=c++17 -c main.cpp
g++ -Wall -std=c++17 -c array-sequence.cpp
g++ -Wall -std=c++17 -o exec_arraySequence main.o array-sequence.o

nelso@Ryan UCRT64 /c/Projetos/cpp/projeto-tads-nelson-vasconcelos-edb1-2026.1/array-sequence
$ make run
./exec_arraySequence
----- ARRAY SEQUENCE : TESTE -----

Capacidade atual: 2

Adicionando valores 10, 20, 30, 40 a sequencia
A V I S O: Array foi realocado e redimensionado com a capacidade: 4
[10] [20] [30] [40]

Adicionando valor 5 ao inicio da sequencia:
A V I S O: Array foi realocado e redimensionado com a capacidade: 8
[5] [10] [20] [30] [40]

Adicionando valor 25 no meio da sequencia:
[5] [10] [25] [20] [30] [40]

Removendo o valor 20 da sequencia:
[5] [10] [25] [30] [40]

A posicao do valor 40 na sequencia eh: 4
```

## **7. Conclusão**

A realização deste trabalho permitiu consolidar de forma prática os conceitos de alocação dinâmica, modularização e manipulação de ponteiros na linguagem C++. O desenvolvimento manual dos Tipos Abstratos de Dados, sem o recurso a bibliotecas prontas da STL, se mostrou um desafio e exigiu um planejamento rigoroso da arquitetura do código e da movimentação dos dados nos vetores.

A implementação destas estruturas demonstrou a importância do controle do tempo de execução e da gestão eficiente da memória. O projeto cumpriu todos os requisitos estipulados, que resultaram num código modular, seguro contra fugas de memória e matematicamente de acordo com as regras dos TADs abordados.